

Tutorial: Quantum Algorithms Implementation in PennyLane/Qiskit

(Session 6 – Day 2, 4 July 2026)

Dr. Shashank Gupta

Learning Objectives

- Learn how to implement basic quantum circuits using PennyLane and Qiskit.
- Implement the Hadamard test and measure expectation values.
- Implement Deutsch–Jozsa algorithm and observe the oracle output.
- Implement Grover’s search algorithm for a small search space.
- Understand how to run circuits on simulators and interpret results.
- Gain hands-on experience with quantum libraries and visualization tools.

1 Setting Up the Environment

Before diving into code, ensure you have Python 3.9+ installed and install the required packages:

```
1 pip install pennylane qiskit matplotlib numpy
```

For hardware-accelerated simulation, you may also install `torch` or `tensorflow` as backends for PennyLane.

2 Hello Quantum World: Hadamard Test

We start with a simple circuit that prepares a superposition and measures the expectation value of Pauli X.

```
1 import pennylane as qml
2 from pennylane import numpy as np
3
4 dev = qml.device('default.qubit', wires=1)
5
6 @qml.qnode(dev)
7 def circuit():
8     qml.Hadamard(wires=0)
9     return qml.expval(qml.PauliX(0))
10
11 print(" X =", circuit()) # Should be close to 1.0
```

The Hadamard gate puts the qubit in the $|+\rangle$ state, giving $\langle X \rangle = 1$.

3 Deutsch–Jozsa Algorithm

We implement the Deutsch–Jozsa algorithm for a 2-bit function (constant or balanced).

```
1 import pennylane as qml
2 from pennylane import numpy as np
3
4 def oracle(choice):
5     """choice = 0 for constant zero, 1 for constant one, 2 for balanced (x0), 3 for
6     balanced (x0 xor x1)"""
7     if choice == 0: # constant 0
8         pass # do nothing
9     elif choice == 1: # constant 1
10        qml.PauliX(wires=2) # flip ancilla to |1>
11    elif choice == 2: # balanced: f(x) = x0
12        qml.CNOT(wires=[0, 2])
13    elif choice == 3: # balanced: f(x) = x0 xor x1
14        qml.CNOT(wires=[0, 2])
15        qml.CNOT(wires=[1, 2])
16
17 dev = qml.device('default.qubit', wires=3)
18 @qml.qnode(dev)
19 def dj_circuit(choice):
20     # Initialize input qubits in |+> and ancilla in |->
21     for i in range(2):
22         qml.Hadamard(wires=i)
23     qml.PauliX(wires=2)
24     qml.Hadamard(wires=2)
25
26     oracle(choice)
27
28     # Apply Hadamard to input qubits again
29     for i in range(2):
30         qml.Hadamard(wires=i)
31
32     return [qml.probs(wires=i) for i in range(2)]
33
34 # Test constant zero
35 probs = dj_circuit(0)
36 print("Constant 0 -> prob|0> on qubit 0:", probs[0][0]) # should be 1
```

If the function is constant, measuring the first two qubits yields $|00\rangle$ with certainty; if balanced, a different outcome appears.

4 Grover's Search Algorithm (2 qubits)

We search for the marked state $|11\rangle$ among four possibilities.

```
1 import pennylane as qml
2 from pennylane import numpy as np
3
4 dev = qml.device('default.qubit', wires=2)
5
6 def oracle_marked():
7     # Marks |11> by applying a Z gate to both qubits (equivalent to controlled-Z)
8     qml.CZ(wires=[0, 1])
9
```

```

10 def diffuser():
11     # Diffusion operator: 2|s><s| - I
12     for i in range(2):
13         qml.Hadamard(wires=i)
14         qml.PauliX(wires=0)
15         qml.PauliX(wires=1)
16         qml.CZ(wires=[0, 1])
17         qml.PauliX(wires=0)
18         qml.PauliX(wires=1)
19     for i in range(2):
20         qml.Hadamard(wires=i)
21
22 @qml.qnode(dev)
23 def circuit():
24     # Initialize uniform superposition
25     for i in range(2):
26         qml.Hadamard(wires=i)
27     # Apply Grover iteration (optimal for N=4 is 1 iteration)
28     oracle_marked()
29     diffuser()
30     return qml.probs(wires=[0, 1])
31
32 probs = circuit()
33 print("Measurement probabilities:", probs)
34 # Expect highest probability for |11> (index 3)

```

After one iteration, the probability of measuring $|11\rangle$ should be near 1.

5 Same Examples in Qiskit

Qiskit offers a different syntax but similar concepts.

```

1 from qiskit import QuantumCircuit, Aer, execute
2 from qiskit.visualization import plot_histogram
3
4 # Deutsch-Jozsa in Qiskit (constant oracle)
5 qc = QuantumCircuit(3, 2) # 2 input qubits, 1 ancilla, 2 classical bits for
    measurement
6 qc.h([0,1])
7 qc.x(2)
8 qc.h(2)
9 # Oracle: constant 0 -> nothing
10 qc.h([0,1])
11 qc.measure([0,1], [0,1])
12
13 backend = Aer.get_backend('qasm_simulator')
14 result = execute(qc, backend, shots=1024).result()
15 counts = result.get_counts()
16 print(counts)
17 plot_histogram(counts)

```

For Grover's algorithm with Qiskit:

```

1 from qiskit import QuantumCircuit, Aer, execute
2 from qiskit.visualization import plot_histogram
3
4 qc = QuantumCircuit(2, 2)
5 qc.h([0,1])
6 # Oracle for |11>

```

```

7 qc.cz(0,1)
8 # Diffusion
9 qc.h([0,1])
10 qc.x([0,1])
11 qc.h(1)
12 qc.cx(0,1)
13 qc.h(1)
14 qc.x([0,1])
15 qc.h([0,1])
16 qc.measure([0,1], [0,1])
17
18 result = execute(qc, Aer.get_backend('qasm_simulator'), shots=1024).result()
19 counts = result.get_counts()
20 print(counts)
21 plot_histogram(counts)

```

6 Visualizing Circuits

Both libraries allow drawing the circuit:

```

1 # PennyLane
2 draw = qml.draw(circuit)
3 print(draw())
4
5 # Qiskit
6 print(qc.draw())

```

7 Running on Hardware (Optional)

If you have access to IBM Quantum hardware, you can replace `Aer.get_backend('qasm_simulator')` with a real device provider after loading your account.

8 Take-away Messages

1. PennyLane provides a seamless interface for quantum machine learning and hybrid computations, while Qiskit offers a rich set of tools for circuit design and execution.
2. Both libraries support simulation and (with proper credentials) execution on real quantum processors.
3. Understanding the underlying quantum operations (gates, oracles, diffusion) is essential regardless of the SDK.
4. Visualization and debugging tools (circuit drawing, state-dumper, histograms) help verify correctness.

Further reading (optional)

PennyLane Documentation: <https://pennylane.ai/qml/>

Qiskit Textbook: <https://qiskit.org/textbook/>

Mario et al., “PennyLane: Automatic differentiation of hybrid quantum-classical computations”, *arXiv:1811.04968*.